

AI-Driven E-Commerce (Master's Level)
Book Structure & 14-Week Plan (Placeholder)

Faculty of Computers and Information Sciences
Mansoura University, Egypt

March 23, 2026

Contents

1	Weekly Plan Overview	17
2	An Introduction to E-Commerce with a Focus on Machine Learning, Deep Learning, and Artificial Intelligence	19
2.1	Introduction to E-Commerce	19
2.1.1	E-Commerce Business Models and Value Chain	20
2.2	Foundations of AI, Machine Learning, and Deep Learning	20
2.2.1	Definitions and Relationships	20
2.2.2	Types of Learning and E-Commerce Examples	21
2.3	Machine Learning Across the E-Commerce Lifecycle	21
2.3.1	Personalized Recommendations	21
2.3.2	Search and Ranking	21
2.3.3	Segmentation, churn, and CLV	21
2.3.4	Dynamic Pricing and Promotion Optimization	21
2.3.5	Demand Forecasting and Inventory Management	22
2.3.6	Fraud Detection and Transaction Security	22
2.3.7	Customer Service and Conversational Commerce	22
2.4	Deep Learning in E-Commerce	22
2.4.1	Neural Recommendation Systems	22
2.4.2	NLP for Product Text and Reviews	22
2.4.3	Computer Vision and Visual Search	22
2.5	Data, Architecture, and Deployment Considerations	22
2.6	Challenges and Responsible AI	23

2.7	Multiple-choice questions (MCQs)	23
3	Platforms, Marketplaces, and Business Models (Week 2)	25
3.1	Learning objectives	25
3.2	From storefront to platform	25
3.3	Value creation, value capture, and growth	25
3.3.1	Value creation and matching	25
3.3.2	Value capture models	26
3.3.3	Network effects	26
3.4	Governance and trust as system requirements	26
3.5	Business model patterns (and architectural implications)	26
3.5.1	B2C retail	26
3.5.2	B2B commerce	27
3.5.3	Marketplace	27
3.5.4	Digital services (subscriptions)	27
3.6	Where AI fits (system-level view)	27
3.7	Hands-on (placeholder)	27
3.8	Case studies (placeholder)	27
3.9	Multiple-choice questions (MCQs)	28
4	E-Commerce Data Foundations: Events, Experimentation, and Metrics (Week 3)	29
4.1	Learning objectives	29
4.2	Why data foundations matter	29
4.3	From business process to event model	30
4.3.1	Events versus state	30
4.3.2	Event taxonomies	31
4.4	Event schema design	31
4.4.1	Required design principles	31
4.4.2	Example event record	32
4.5	Identity resolution and entity modeling	32

4.5.1	Identifiers are not interchangeable	33
4.5.2	Identity stitching	33
4.5.3	Privacy-aware identity design	33
4.6	Data quality as a product requirement	34
4.6.1	Data contracts	34
4.6.2	Validation and observability	34
4.7	Attribution and its limits	35
4.7.1	Common attribution models	35
4.7.2	Selection bias and measurement bias	35
4.8	Metrics layers and decision quality	36
4.8.1	North Star and supporting metrics	36
4.8.2	Metric dimensions	36
4.8.3	Retention and cohort logic	37
4.9	Experimentation in e-commerce	37
4.9.1	A/B testing fundamentals	37
4.9.2	Common experimental pitfalls	37
4.9.3	Bandits and adaptive experimentation	38
4.10	From analytics to machine learning features	38
4.11	Hands-on lab: build a minimal metrics layer	38
4.11.1	Lab scenario	39
4.11.2	Lab tasks	39
4.11.3	Expected learning outcome	39
4.12	Managerial and architectural implications	39
4.13	Multiple-choice questions (MCQs)	40
4.14	Exercises (short)	41
4.15	Mini-case (Odoo-linked, vendor-neutral)	41
5	Enterprise Architecture for E-Commerce (Week 4)	43
5.1	Learning objectives	43
5.2	Why enterprise architecture matters in e-commerce	43

5.3	Enterprise architecture as a set of views	44
5.4	Capability maps	44
5.4.1	Core e-commerce capabilities	45
5.4.2	Why capability maps are useful	45
5.5	Value streams	45
5.5.1	Typical e-commerce value streams	46
5.5.2	Why value streams matter architecturally	46
5.6	Domain boundaries and bounded contexts	46
5.6.1	Typical commerce domains	46
5.6.2	Good boundaries versus bad boundaries	47
5.7	Systems of record and systems of engagement	47
5.7.1	Typical patterns	47
5.7.2	Systems of engagement	48
5.8	Reference architecture for AI-driven e-commerce	48
5.8.1	Where AI fits	48
5.9	Current state, target state, and transition architecture	49
5.9.1	Current-state analysis	49
5.9.2	Target architecture	49
5.9.3	Transition architecture	49
5.10	Architectural concerns specific to AI	50
5.11	Operating model and team topology	50
5.11.1	Typical team patterns	50
5.12	Artifacts for students and architects	50
5.13	Hands-on artifacts	51
5.14	Managerial implications	51
5.15	Multiple-choice questions (MCQs)	52
5.16	Exercises (short)	53
5.17	Mini-case (Odoo-linked, vendor-neutral)	53

6.1	Scope (placeholder)	55
6.2	Hands-on lab (placeholder)	55
6.3	Multiple-choice questions (MCQs)	55
6.4	Exercises (short)	56
6.5	Mini-case (Odoo-linked, vendor-neutral)	57
7	ERP/CRM/SCM Integration in Commerce (Week 6)	59
7.1	Scope (placeholder)	59
7.2	Artifacts (placeholder)	59
7.3	Multiple-choice questions (MCQs)	59
7.4	Exercises (short)	60
7.5	Mini-case (Odoo-linked, vendor-neutral)	61
8	Recommenders I: Classical Methods + Learning-to-Rank (Week 7)	63
8.1	Scope (placeholder)	63
8.2	Hands-on lab (placeholder)	63
9	Recommenders II: Deep Learning, Embeddings, and Retrieval (Week 8)	65
9.1	Scope (placeholder)	65
9.2	Hands-on lab (placeholder)	65
10	Pricing and Promotions with ML + Optimization (Week 9)	67
10.1	Scope (placeholder)	67
10.2	Hands-on lab (placeholder)	67
11	Fraud, Risk, and Trust & Safety (Week 10)	69
11.1	Scope (placeholder)	69
11.2	Hands-on lab (placeholder)	69
12	Customer Service Automation with LLMs (Week 11)	71
12.1	Scope (placeholder)	71
12.2	Hands-on lab (placeholder)	71

13 MLOps + DataOps for E-Commerce (Week 12)	73
13.1 Scope (placeholder)	73
13.2 Artifacts (placeholder)	73
14 Security, Privacy, Compliance, and Responsible AI (Week 13)	75
14.1 Scope (placeholder)	75
14.2 Discussion (placeholder)	75
15 Capstone: Enterprise-Grade AI E-Commerce Architecture (Week 14)	77
15.1 Scope (placeholder)	77
15.2 Capstone deliverables (placeholder)	77
A Datasets and Synthetic Data for Teaching (placeholder)	79
B Template: System Design Document (placeholder)	81
C Template: Data Contract (placeholder)	83
D Template: Model Card and Risk Assessment (placeholder)	85

How to Use This Draft

This document is a *structure-only* placeholder for an AI-driven, practical Master's-level e-commerce book. Each chapter will be written later; for now, sections contain brief bullets stating scope, key skills, and deliverables.

Intended Audience and Prerequisites

- Audience: Master's students in Information Systems / Computer & Information Sciences.
- Prereqs (recommended): databases, basic programming, web fundamentals, and introductory ML.
- Tooling (suggested): Python, notebooks, APIs, cloud services, and an ERP/EA modeling tool (as available).

Hands-on Track (Agreed)

- Approach: both tracks (lighter each).
- Track A (Enterprise/ERP): Odoo-focused integration labs (Sales/Inventory/Accounting flows).
- Track B (AI/ML): ML-heavy notebooks using e-commerce datasets (recsys, pricing, fraud, support).

Course Outcomes (Draft)

- Design end-to-end e-commerce systems with enterprise architecture (EA) and integration concerns.
- Apply ML/DL/AI to personalization, search, pricing, fraud, operations, and customer service.
- Integrate e-commerce with ERP/CRM/SCM (reference: Odoo) via APIs, events, and modern integration patterns.
- Evaluate systems for security, privacy, governance, ethics, and measurable business value.

Chapter 1

Weekly Plan Overview

Course emphasis (agreed)

- Overall emphasis: balanced (EA/ERP + ML/DL/AI).
- ERP reference stack: Odoo (concepts remain vendor-neutral where possible).

Week	Topic (maps to the corresponding chapter)
1	Foundations of modern e-commerce systems and AI-driven product thinking
2	Digital platforms, marketplaces, and business models (B2C/B2B/D2C, multi-sided platforms)
3	E-commerce data foundations: events, tracking, data quality, and experimentation
4	Enterprise architecture for e-commerce: capabilities, domains, and reference architectures
5	Enterprise integration: APIs, iPaaS, ESB, events/streaming, and integration patterns
6	ERP/CRM/SCM integration: order-to-cash, procure-to-pay, inventory, and master data
7	Recommenders I: classical + learning-to-rank for personalization and search
8	Recommenders II: deep learning, embeddings, sequence models, and retrieval
9	Pricing and promotion analytics: forecasting, elasticity, and optimization (ML + OR)
10	Fraud, risk, and trust & safety: anomaly detection, graph ML, and AML patterns
11	Customer service automation: LLMs, RAG, chatbots, and agentic workflows
12	MLOps + DataOps for e-commerce: deployment, monitoring, drift, and governance
13	Security, privacy, compliance, and responsible AI in commerce
14	Capstone: enterprise-grade AI e-commerce solution architecture + evaluation

Chapter 2

An Introduction to E-Commerce with a Focus on Machine Learning, Deep Learning, and Artificial Intelligence

Abstract

Electronic commerce (e-commerce) has evolved from static online catalogs into highly dynamic, data-driven ecosystems. This transformation has been powered largely by advances in artificial intelligence (AI), particularly machine learning (ML) and deep learning (DL). This chapter introduces the foundations of e-commerce and explains how AI techniques are embedded across the e-commerce value chain—from customer acquisition and product discovery to pricing, logistics, and fraud prevention. The discussion is aimed at master’s students in computer and information sciences, and therefore emphasizes conceptual clarity, system-level thinking, and the mapping between theoretical models and real-world e-commerce applications.

Keywords: e-commerce, artificial intelligence, machine learning, deep learning, recommendation systems, dynamic pricing, fraud detection, personalization, logistics optimization

2.1 Introduction to E-Commerce

E-commerce refers to the buying and selling of goods and services via electronic networks, primarily the internet [?]. It encompasses a broad set of transactional models: business-to-consumer (B2C) online retail, business-to-business (B2B) procurement platforms, consumer-to-consumer (C2C) marketplaces, and consumer-to-business (C2B) models such as influencer platforms and freelance marketplaces [?].

From a computing perspective, an e-commerce platform is not just a web front-end. It is a socio-technical system composed of:

- User interfaces (web, mobile, conversational)

- Application logic (catalog, cart, checkout, account management)
- Payment and risk engines
- Logistics and fulfillment systems
- Data infrastructure and analytics pipelines
- AI/ML components that drive personalization, prediction, and automation

A key characteristic of e-commerce is its data richness. Every user interaction—page views, searches, clicks, scrolls, wishlists, add-to-carts, purchases, returns, and even time-to-decision—can be captured as fine-grained behavioral logs. Combined with product metadata, prices, promotions, and external contextual data, this creates an ideal environment for AI and ML [? ?].

2.1.1 E-Commerce Business Models and Value Chain

Common e-commerce models include:

- **B2C retail:** Online stores selling directly to consumers.
- **Marketplaces:** Platforms mediating between many sellers and buyers.
- **B2B platforms:** Portals for wholesale ordering, procurement, and supply chain integration.
- **Digital services:** Software subscriptions, digital content, and online education.

Across these models, the **e-commerce value chain** typically includes:

1. Customer acquisition and traffic generation (marketing, ads, SEO)
2. Product discovery and decision support (search, recommendations, reviews)
3. Conversion and transaction processing (checkout, payments, risk checks)
4. Order fulfillment and logistics (inventory, warehousing, routing)
5. Post-purchase engagement (support, returns, loyalty, re-activation)

AI, ML, and DL are now present at each stage of this chain [?].

2.2 Foundations of AI, Machine Learning, and Deep Learning

2.2.1 Definitions and Relationships

- **Artificial Intelligence (AI):** Systems that exhibit behaviors considered “intelligent” (perception, reasoning, learning, decision-making).

- **Machine Learning (ML):** Algorithms that learn patterns from data and improve with experience.
- **Deep Learning (DL):** ML methods based on multi-layer neural networks that learn representations from large-scale data.

Thus, deep learning $\subset$ machine learning $\subset$ artificial intelligence [? ? ?].

2.2.2 Types of Learning and E-Commerce Examples

1. **Supervised learning:** learn $f : X \rightarrow Y$ from labeled examples (x_i, y_i) . Examples: conversion prediction, fraud classification, demand forecasting.
2. **Unsupervised learning:** discover structure in unlabeled data. Examples: customer segmentation, anomaly detection, and learning embeddings from interactions.
3. **Reinforcement learning:** learn a policy to maximize long-term reward. Examples: recommendation policies and dynamic pricing under constraints.
4. **Representation learning:** learn features $\phi(x)$ automatically (often via DL). Examples: user/item embeddings; Transformer-based session representations.

2.3 Machine Learning Across the E-Commerce Lifecycle

2.3.1 Personalized Recommendations

Recommendation systems use past behavior and product attributes to suggest items a user is likely to buy [? ?]. Key evaluation ideas include ranking quality (precision@ k , recall@ k , NDCG) and business impact (incremental revenue) [?].

2.3.2 Search and Ranking

Learning-to-rank uses user feedback (including clicks) to optimize product ranking in response to queries [?]. Modern systems increasingly incorporate semantic retrieval and NLP components [?].

2.3.3 Segmentation, churn, and CLV

Segmentation (e.g., via clustering) and predictive modeling (e.g., churn/CLV) connect behavioral data to marketing and retention actions [?].

2.3.4 Dynamic Pricing and Promotion Optimization

Pricing combines prediction (demand response) with optimization under constraints; evaluation typically requires controlled online experimentation [? ?].

2.3.5 Demand Forecasting and Inventory Management

Forecasting methods range from statistical time-series models to ML sequence models; forecasts drive replenishment and inventory decisions [? ?].

2.3.6 Fraud Detection and Transaction Security

Fraud detection is an imbalanced classification problem with non-stationarity (attackers adapt), and must balance false positives against fraud loss [? ?].

2.3.7 Customer Service and Conversational Commerce

Customer support automation mixes information retrieval, intent/entity extraction, and increasingly LLM-based interaction [?].

2.4 Deep Learning in E-Commerce

DL is especially useful for unstructured and high-dimensional data (text, images, sequences) [?].

2.4.1 Neural Recommendation Systems

Deep recommender architectures model non-linear user–item interactions and session dynamics [? ?].

2.4.2 NLP for Product Text and Reviews

NLP supports query understanding, semantic search, review analysis, and conversational interfaces [?].

2.4.3 Computer Vision and Visual Search

Vision models support category classification, attribute extraction, and similarity search via embeddings [?].

2.5 Data, Architecture, and Deployment Considerations

E-commerce ML requires robust pipelines (data quality, governance), reliable deployment (latency, availability), and continuous evaluation (monitoring and experiments) [?].

2.6 Challenges and Responsible AI

Key challenges include bias, privacy/security, explainability for high-impact decisions, and the organizational processes needed for safe deployment [? ?].

2.7 Multiple-choice questions (MCQs)

1. Which relationship is correct?
 - (a) $AI \subset ML \subset DL$
 - (b) $DL \subset ML \subset AI$
 - (c) $ML \subset DL \subset AI$
 - (d) AI, ML, and DL are disjoint fields
2. In e-commerce, which is **most** naturally framed as a *ranking* problem?
 - (a) Choosing the best warehouse location for a new facility
 - (b) Ordering products in a search results page for a query
 - (c) Estimating the total demand for next quarter
 - (d) Balancing accounting entries for an invoice
3. Which metric is **most** aligned with offline evaluation of a top- k recommender?
 - (a) $NDCG@k$
 - (b) Mean squared error (MSE) on prices
 - (c) Page load time (ms)
 - (d) Uptime percentage
4. Fraud detection is often difficult because:
 - (a) Fraud labels are always perfectly accurate
 - (b) The problem is typically extremely imbalanced and attackers adapt over time
 - (c) Fraud has no economic cost, only technical cost
 - (d) The best approach is always rule-based
5. Which statement best describes why A/B testing is important in e-commerce ML?
 - (a) It guarantees the model is unbiased
 - (b) It measures causal impact on business KPIs under real user behavior
 - (c) It replaces the need for offline evaluation entirely
 - (d) It eliminates the need for monitoring after deployment

Answer key

1. (b)
2. (b)
3. (a)
4. (b)
5. (b)

Chapter 3

Platforms, Marketplaces, and Business Models (Week 2)

3.1 Learning objectives

- Distinguish e-commerce *channels* (storefronts) from *platforms* (ecosystems) and *marketplaces* (multi-seller).
- Analyze platform economics: network effects, multi-homing, and pricing of participation.
- Translate business model choices into architectural requirements (identity, trust, data, integration, ERP).

3.2 From storefront to platform

- **Storefront:** a single merchant selling to buyers (B2C/B2B) via a web/app channel.
- **Platform:** a system that enables interactions among multiple participant groups (e.g., buyers, sellers, advertisers, logistics providers).
- **Marketplace:** a platform where multiple sellers list items and transactions are mediated by platform policies and infrastructure.

3.3 Value creation, value capture, and growth

3.3.1 Value creation and matching

Platforms create value by reducing search and transaction costs, increasing variety, and improving trust; AI frequently strengthens these effects by improving matching and decision support.

3.3.2 Value capture models

- Take rate (commission on completed transactions)
- Subscription (seller or buyer membership tiers)
- Listing fees and value-added services (e.g., promotions, analytics)
- Advertising (sponsored listings, retail media)
- Fulfillment fees (warehousing, last-mile delivery, returns handling)

3.3.3 Network effects

- **Direct network effects:** more users attract more users (e.g., social commerce communities).
- **Indirect network effects:** more buyers attract more sellers and vice versa (classic marketplace dynamic).
- **Cold start:** bootstrapping supply and demand when network effects are weak.
- **Multi-homing:** sellers/buyers participate in multiple platforms; affects differentiation strategy.

3.4 Governance and trust as system requirements

Marketplace governance is not “policy only”: it becomes a set of product and system requirements.

- Seller onboarding, verification, and compliance workflows (KYC/KYB where relevant).
- Catalog integrity: listing policies, moderation, and counterfeit detection.
- Reviews/ratings, dispute resolution, refunds/returns policy enforcement.
- Trust & safety operations: rule+ML detection, case management, and human-in-the-loop escalation.

3.5 Business model patterns (and architectural implications)

3.5.1 B2C retail

- Key systems: PIM/catalog, promotions, payments, fulfillment, customer support.
- ERP/CRM integration: order-to-cash, inventory, accounting, returns.

3.5.2 B2B commerce

- Quotes, negotiated pricing, approval workflows, invoicing, credit limits.
- Integration-heavy: procurement, supplier management, and EDI/API flows.

3.5.3 Marketplace

- Split catalog ownership (platform vs sellers), seller tools, payout/settlement.
- Additional “control plane”: seller policies, listing moderation, abuse prevention.

3.5.4 Digital services (subscriptions)

- Recurring billing, entitlements, and churn/retention analytics.
- Usage-based pricing requires event design and metering.

3.6 Where AI fits (system-level view)

- **Matching layer:** search, recommendations, ranking.
- **Trust layer:** fraud/abuse detection, content moderation.
- **Operations layer:** forecasting, replenishment, routing.
- **Experience layer:** personalization and conversational commerce.

3.7 Hands-on (placeholder)

- Create a one-page *platform blueprint*: participant groups, value exchange, pricing model, and required capabilities.
- Map capabilities to an EA view and identify which capabilities are ERP-owned vs commerce-owned.

3.8 Case studies (placeholder)

- Modern marketplace patterns (generic; no vendor lock-in).

3.9 Multiple-choice questions (MCQs)

1. Which statement best distinguishes a *storefront* from a *marketplace*?
 - (a) A storefront is always B2B; a marketplace is always B2C.
 - (b) A storefront has a single merchant of record; a marketplace mediates transactions among multiple sellers and buyers.
 - (c) A storefront cannot use AI; a marketplace must use AI.
 - (d) A storefront requires ERP integration; a marketplace does not.
2. *Indirect network effects* in marketplaces most directly mean:
 - (a) The platform has a single user group and growth is linear.
 - (b) The value to buyers increases as more sellers join, and the value to sellers increases as more buyers join.
 - (c) Users prefer multiple platforms (multi-homing) regardless of differentiation.
 - (d) The platform must subsidize logistics to succeed.
3. In B2B commerce, which requirement is **most typical** compared to B2C?
 - (a) Session-based recommendations
 - (b) Negotiated pricing and approval workflows
 - (c) Visual search
 - (d) Social reviews and influencer marketing
4. *Multi-homing* refers to:
 - (a) Hosting the platform in multiple cloud regions.
 - (b) Sellers or buyers participating in multiple platforms simultaneously.
 - (c) Using multiple payment gateways for redundancy.
 - (d) Maintaining multiple warehouses for the same SKU.
5. Which is the **best** example of a marketplace *governance* capability?
 - (a) Re-ranking items using embeddings
 - (b) Seller onboarding verification and dispute resolution policies
 - (c) Demand forecasting for replenishment
 - (d) Offline model training with hyperparameter tuning

Answer key

1. (b)
2. (b)
3. (b)
4. (b)
5. (b)

Chapter 4

E-Commerce Data Foundations: Events, Experimentation, and Metrics (Week 3)

4.1 Learning objectives

- Design an event-driven data model for storefront, marketplace, and mobile commerce scenarios.
- Distinguish identifiers, identities, sessions, and entities across operational and analytical systems.
- Explain how attribution, data quality, and metric definitions affect decision-making and ML performance.
- Plan controlled experiments and understand when adaptive approaches such as bandits are appropriate.

4.2 Why data foundations matter

AI systems in e-commerce are only as reliable as the data substrate beneath them. Recommendation models, pricing systems, fraud detectors, and executive dashboards all depend on the same underlying facts: who did what, when, in which context, and with what downstream outcome. If those facts are delayed, duplicated, ambiguously defined, or disconnected across systems, both analytics and machine learning become fragile.

This chapter treats data foundations as a *system design* problem rather than a reporting problem. The goal is not merely to “collect more data,” but to create a coherent measurement layer that can support:

- Product analytics and funnel analysis
- Operational reporting across commerce and ERP systems

- Reliable feature generation for ML models
- Controlled online experimentation
- Governance, auditing, and cross-functional alignment

4.3 From business process to event model

E-commerce platforms contain several interacting process layers:

- **Customer interaction layer:** impressions, searches, clicks, scrolls, add-to-cart, checkout steps, support interactions.
- **Commercial transaction layer:** order placement, payment authorization, invoicing, shipment, cancellation, return, refund.
- **Operational layer:** inventory adjustments, picking, packing, supplier replenishment, carrier status changes.
- **Experimentation layer:** exposure to treatments, eligibility checks, metric assignment, and holdouts.

Each layer emits events, but those events serve different purposes. A page view event captures human behavior. An order-created event captures a business transaction. A shipment-delivered event captures operational state. An experiment-exposed event captures causal assignment context. Strong data design begins by separating these concerns while still linking them with stable keys.

4.3.1 Events versus state

Students often confuse *events* with *tables of current state*. The distinction is fundamental:

- **State** answers “what is true now?” Examples: current inventory, current product price, current loyalty tier.
- **Events** answer “what happened?” Examples: product viewed, cart updated, payment failed, order shipped.

State is necessary for operations, but events are essential for causality, sequence modeling, root-cause analysis, and reconstructing past behavior. For example, a fraud model may need the sequence of failed payment attempts before an order was placed, not just the final order record. Similarly, retention analysis depends on when a user returned, not merely whether the user is currently “active.”

4.3.2 Event taxonomies

A useful event taxonomy groups events into classes that are stable even when the user interface changes. For a commerce platform, a practical taxonomy may include:

- **Discovery events:** homepage viewed, category viewed, search submitted, filter applied, result clicked.
- **Consideration events:** product viewed, review expanded, comparison opened, wishlist updated.
- **Intent events:** add-to-cart, remove-from-cart, coupon applied, shipping option selected.
- **Transaction events:** checkout started, payment submitted, order completed, order canceled.
- **Post-purchase events:** shipment delivered, return requested, refund issued, support ticket opened.

This style of taxonomy makes analytics more robust than mirroring UI element names directly. If a button label changes from “Buy Now” to “Checkout,” the business meaning should still map to a stable event family rather than creating an entirely new measurement definition.

4.4 Event schema design

An event is only analytically useful if its schema is consistent. At minimum, most commerce events should include:

- **Event name:** stable business-oriented label such as `product_viewed` or `checkout_started`
- **Event time:** when the event actually occurred
- **Ingestion time:** when the event reached the data platform
- **Actor identifiers:** user ID, guest ID, device ID, session ID, seller ID where relevant
- **Entity identifiers:** product ID, SKU ID, cart ID, order ID, payment ID
- **Context fields:** channel, app version, locale, campaign, experiment assignment, page type
- **Properties:** quantity, price, discount, position in ranking, search query, payment method

4.4.1 Required design principles

- **Business semantics before UI semantics:** schemas should reflect domain meaning, not implementation accidents.
- **Stable field names:** renaming fields casually breaks dashboards, pipelines, and models.
- **Explicit units:** store price currency, quantity units, and time zones unambiguously.

- **Immutable raw events:** avoid rewriting the historical log except for tightly governed correction procedures.
- **Versioned schemas:** consumers must know when a producer changed meaning or structure.

4.4.2 Example event record

The following conceptual record shows how a product view may be represented:

```

event_name      = product_viewed
event_time      = 2026-03-23T14:03:11Z
ingestion_time  = 2026-03-23T14:03:13Z
user_id         = null
guest_id        = g_1842
session_id      = s_88fa
product_id      = p_501
sku_id          = sku_501_bl_m
page_type       = product_detail
channel         = mobile_app
campaign_id     = spring_push_7
rank_position   = 4
experiment_id   = rec_widget_v3
variant         = treatment

```

The example illustrates an important practice: authenticated and anonymous identifiers may coexist. This supports both guest behavior analysis and later identity stitching when a guest eventually logs in or purchases.

4.5 Identity resolution and entity modeling

One of the hardest problems in e-commerce analytics is deciding what a “customer” means across channels and systems. The same person may appear as:

- an anonymous web browser cookie,
- a mobile-app installation,
- a logged-in storefront account,
- a CRM contact,
- an ERP customer or partner record,
- a marketplace buyer account,
- or multiple records created through operational errors.

4.5.1 Identifiers are not interchangeable

Different identifiers serve different roles:

- **User/account ID:** application-level identity for authentication and personalization.
- **Customer/partner ID:** commercial identity used in ERP, billing, and support.
- **Session ID:** short-lived grouping of interactions.
- **Device or browser ID:** technical identifier for anonymous continuity.
- **Order ID:** transaction-level identity.
- **Household or organization ID:** aggregation identity for B2B or family accounts.

If analysts casually join these identifiers as though they were equivalent, metrics become unstable. For example, “active customers” may mean active accounts in one report but active billing entities in another. That inconsistency can distort conversion rates, retention cohorts, and lifetime value estimates.

4.5.2 Identity stitching

Identity stitching is the process of linking records that likely belong to the same real-world actor. Common rules include:

- linking a guest cart to a logged-in account after authentication,
- attaching a mobile device history to a user after sign-in,
- associating storefront orders with an ERP customer once order export completes,
- maintaining a separate unresolved pool when confidence is low.

The core design principle is to preserve raw evidence and build linkage logic transparently. Many downstream problems arise when teams overwrite raw IDs with guessed “master” identities and can no longer audit how the match occurred.

4.5.3 Privacy-aware identity design

Identity resolution must also respect privacy and compliance constraints. A technically convenient identifier is not automatically a legally or ethically appropriate one. Good practice includes:

- minimizing collection of directly identifying information,
- separating analytical IDs from operational personal data where possible,
- documenting retention periods,
- and ensuring that consent and deletion workflows can propagate to analytical systems.

4.6 Data quality as a product requirement

Data quality problems in commerce are rarely abstract. They directly alter revenue reporting, experiment reads, and ML labels. Representative failure modes include:

- **Missing events:** checkout steps are undercounted, making conversion funnels look healthier than reality.
- **Duplicate events:** add-to-cart counts inflate intent metrics and derived model features.
- **Late events:** daily dashboards and training jobs disagree because the data arrive after aggregation windows close.
- **Semantic drift:** an event name remains unchanged while the product team silently changes its meaning.
- **Broken joins:** product IDs in clickstream logs no longer match product IDs in the catalog or ERP.

4.6.1 Data contracts

An effective way to reduce these failures is to define *data contracts* between producers and consumers. A data contract specifies:

- event name and schema,
- field definitions and allowed values,
- freshness expectations,
- ownership and escalation path,
- versioning policy,
- and validation rules.

This shifts governance left. Instead of discovering breakage after dashboards fail, producers must acknowledge downstream dependencies before changing event payloads.

4.6.2 Validation and observability

Data platforms should monitor more than pipeline uptime. Useful checks include:

- row-count anomalies by event family,
- null-rate spikes in required fields,
- duplicate-rate thresholds by event key,

- cross-system reconciliation between orders, payments, and shipments,
- and schema drift alerts when producers add or remove fields.

In mature teams, these checks are treated as production monitoring. Broken analytics should be considered an operational incident, not a minor inconvenience.

4.7 Attribution and its limits

Attribution asks which touchpoints influenced an outcome such as purchase, repeat visit, or subscription renewal. In e-commerce, this sounds straightforward but is often confounded by user behavior, cross-device journeys, channel interactions, and pre-existing intent.

4.7.1 Common attribution models

- **Last-touch attribution:** credits the final recorded touchpoint before conversion.
- **First-touch attribution:** credits the earliest touchpoint in the conversion path.
- **Multi-touch heuristics:** divide credit across several interactions.
- **Incrementality-based approaches:** estimate causal lift rather than merely assigning credit.

Heuristic attribution is useful for operational reporting, but it should not be confused with causal inference. A campaign that appears near the end of many journeys may receive last-touch credit even if it did not truly create incremental demand.

4.7.2 Selection bias and measurement bias

Attribution systems are affected by:

- **Selection bias:** users exposed to an experience may differ systematically from users who are not.
- **Survivorship bias:** only recorded users or channels appear in the analysis.
- **Cross-device loss:** a user researches on one device and purchases on another without a reliable link.
- **Channel interference:** paid and organic channels interact, making isolated credit assignments misleading.

This is why attribution reporting is a descriptive tool, whereas experimentation is the primary causal tool.

4.8 Metrics layers and decision quality

Executives often ask for “the conversion rate” as though it were a single obvious number. In reality, metric definitions encode business rules. Without a shared metrics layer, each team may compute the same KPI differently.

4.8.1 North Star and supporting metrics

Many commerce organizations choose a high-level outcome metric such as gross merchandise value, contribution margin, or retained active buyers. That metric must then be decomposed into leading indicators:

- traffic volume,
- product-detail engagement,
- add-to-cart rate,
- checkout completion rate,
- repeat purchase rate,
- return rate,
- and support burden.

For educational purposes, it is useful to connect these metrics to event definitions explicitly. For instance:

$$\text{Add-to-cart rate} = \frac{\text{\#sessions with add_to_cart}}{\text{\#sessions with product_view}}$$

This formula is simple, but each term still depends on sessionization rules, bot filtering, and event validity criteria.

4.8.2 Metric dimensions

Commerce metrics usually need to be sliced by:

- channel (web, app, store-assisted, marketplace),
- customer segment (new, returning, VIP, B2B account tier),
- geography and locale,
- device class,
- product category and seller,
- experiment cohort.

A robust metrics layer therefore requires both canonical definitions and dimensional consistency. It should be possible to explain why a number changes, not just to report the number itself.

4.8.3 Retention and cohort logic

Retention metrics are especially sensitive to definition. Possible questions include:

- Did the user return to browse?
- Did the user make another purchase?
- Did the organization place another order from any employee account?
- Did the customer remain active despite returns or failed payments?

Each version may be legitimate, but each reflects a different product and business question. Good analytical practice is to name retention metrics with explicit behavioral criteria rather than using ambiguous shorthand.

4.9 Experimentation in e-commerce

Offline evaluation is valuable, but it cannot fully predict live business impact. Ranking changes alter user behavior. Promotions affect demand patterns. Fraud rules shift attacker behavior. For these reasons, e-commerce systems rely heavily on online experiments.

4.9.1 A/B testing fundamentals

An A/B test compares a treatment against a control by random assignment. The point of randomization is not to guarantee improvement; it is to make the groups comparable in expectation so that observed differences can be interpreted causally.

Key design choices include:

- **Unit of randomization:** user, session, device, seller, or region.
- **Exposure logic:** where and when eligibility is checked.
- **Primary KPI:** the main outcome the experiment is intended to improve.
- **Guardrails:** metrics that protect against harmful side effects such as latency, return rate, or support contacts.
- **Analysis window:** how long outcomes are observed after exposure.

4.9.2 Common experimental pitfalls

- **Contamination:** the same user sees both variants through multiple devices or sessions.
- **Novelty effects:** short-term excitement masks weak long-term performance.
- **Interference:** one user's treatment influences another user, seller, or market.

- **Metric leakage:** the treatment changes how events are logged rather than how users behave.
- **Underpowered tests:** sample sizes are too small to detect meaningful differences.

These pitfalls illustrate why experimentation is partly a data-engineering problem. If exposure logs and outcome metrics do not align, the test result is not credible.

4.9.3 Bandits and adaptive experimentation

Multi-armed bandits are useful when the platform wants to balance learning with near-term reward. Compared with standard A/B tests, bandits typically allocate more traffic to arms that appear to perform better. This can be attractive for:

- promotions on fast-moving catalogs,
- ranking widgets with many interchangeable treatments,
- high-traffic surfaces where opportunity cost is large.

However, bandits are not a universal replacement for A/B testing. They complicate interpretation, can interact badly with delayed rewards, and are less appropriate when decision-makers need a clean causal estimate for a fixed policy choice. In practice, many organizations use A/B tests for clear comparisons and adaptive methods for optimization on mature surfaces.

4.10 From analytics to machine learning features

The same event stream that supports dashboards often becomes the basis for ML features. Examples include:

- recent category views for recommendation,
- failed login or payment attempts for fraud detection,
- coupon usage history for promotion targeting,
- time since last order for churn models,
- return behavior and order defects for seller-quality models.

This creates an important dependency: if event semantics drift, model features drift as well. A model can degrade not only because user behavior changes, but because the measurement pipeline changed underneath it. For this reason, feature definitions should be traceable to governed event definitions.

4.11 Hands-on lab: build a minimal metrics layer

The aim of the lab is to help students bridge raw event logs and business-facing KPIs.

4.11.1 Lab scenario

Assume a storefront emits five core events: `session_started`, `product_viewed`, `add_to_cart`, `checkout_started`, and `order_completed`. Students are given a small synthetic log containing timestamps, session IDs, customer IDs where available, product IDs, and order IDs.

4.11.2 Lab tasks

1. Create a canonical event table with required fields, event validation flags, and ingestion metadata.
2. Build daily metrics for sessions, product views, add-to-cart rate, checkout-start rate, purchase conversion rate, and day-7 repeat purchase.
3. Define at least three data-quality checks and show how failing checks affect one KPI.
4. Create a simple cohort table for first purchase date and compute repeat purchase behavior by cohort.
5. Simulate an experiment exposure field and compute the primary KPI by control versus treatment.

4.11.3 Expected learning outcome

By the end of the lab, students should be able to explain why a metric is not just an aggregation, but the result of event design choices, identity assumptions, and validation rules.

4.12 Managerial and architectural implications

Data foundations are not owned by analysts alone. They require coordination across product, engineering, data, and enterprise systems teams. Architecturally, the following questions matter:

- Which identifiers are authoritative, and where are they mastered?
- Which events belong in the clickstream versus operational event streams?
- How are storefront events linked to ERP objects such as orders, invoices, and shipments?
- What happens when a schema changes in the storefront while ERP integrations remain stable, or vice versa?
- Who approves metric definitions that drive executive dashboards and model targets?

These are governance questions, but they also influence software boundaries. Weak ownership at this layer creates confusion throughout the commerce stack.

4.13 Multiple-choice questions (MCQs)

1. In event-based analytics for e-commerce, which statement is most accurate?
 - (a) Events should be stored without timestamps to save space.
 - (b) Events are most useful when they have a consistent schema, a timestamp, and stable identifiers.
 - (c) Only purchase events matter; click and view events are noise.
 - (d) The best schema changes frequently to match UI updates.
2. Which is an example of a *data quality* problem that can directly harm ML models?
 - (a) A model uses a neural network instead of linear regression.
 - (b) Missing or duplicated events for key actions (e.g., add-to-cart) create biased labels/features.
 - (c) Using SQL instead of Python for ETL.
 - (d) Choosing a smaller batch size during training.
3. Why are offline metrics often insufficient for evaluating an e-commerce ranking model?
 - (a) Offline metrics are always higher than online metrics.
 - (b) Offline metrics cannot measure causal impact under real user behavior and feedback loops.
 - (c) Offline evaluation cannot be computed from logs.
 - (d) Offline metrics are not used in industry.
4. In an A/B test, the primary purpose of randomization is to:
 - (a) Increase model complexity.
 - (b) Ensure the treatment and control groups are comparable in expectation.
 - (c) Eliminate the need for monitoring.
 - (d) Guarantee the KPI improves.
5. A common *identity resolution* challenge in e-commerce is:
 - (a) Mapping product IDs to SKU IDs.
 - (b) Linking the same person across devices/sessions while respecting privacy constraints.
 - (c) Computing NDCG@ k efficiently.
 - (d) Selecting a cloud region.

Answer key

1. (b)
2. (b)
3. (b)
4. (b)
5. (b)

4.14 Exercises (short)

1. Propose an event taxonomy for a storefront: list at least 10 events (search, view, add-to-cart, checkout steps, purchase, return) and for each event specify 3–5 key fields.
2. Define a North Star metric for an e-commerce product and break it into 3–5 leading indicators. Explain how you would compute each from event logs.
3. Design a simple A/B test for a recommendation widget. Specify: unit of randomization, primary KPI, guardrail metrics, and an example of a bias/confounder to watch for.

4.15 Mini-case (Odoo-linked, vendor-neutral)

Scenario: A hybrid B2C/B2B company uses Odoo (Sales, Inventory, Accounting, CRM). The storefront and mobile app generate clickstream events.

Task: Design a minimal data model that supports both analytics and ML:

- Define how you will link clickstream identities to Odoo customers/partners (when possible) and how you handle guests.
- Propose which entities should have stable IDs across systems (customer, product, order, shipment) and which are channel-specific.
- Specify 5 “data contracts” (producer $\rightarrow$ consumer) that reduce breakage when the UI or ERP changes.

Chapter 5

Enterprise Architecture for E-Commerce (Week 4)

5.1 Learning objectives

- Explain how enterprise architecture (EA) helps align business goals, operating models, and technology decisions in e-commerce.
- Use capability maps, value streams, domain boundaries, and system-of-record thinking to structure a commerce platform.
- Distinguish current-state architecture from target architecture and transition planning.
- Analyze how AI capabilities fit into enterprise architecture without turning the architecture into a collection of isolated point solutions.

5.2 Why enterprise architecture matters in e-commerce

E-commerce systems evolve quickly. New channels, promotions, AI features, logistics partnerships, and regulatory constraints are introduced faster than many enterprise systems were originally designed to handle. Without architectural discipline, organizations often accumulate fragmented storefronts, duplicated customer records, inconsistent pricing logic, and analytics pipelines that are disconnected from operational truth.

Enterprise architecture is useful because it provides a way to reason across these layers at once. It connects:

- business strategy,
- organizational capabilities,
- application boundaries,
- data ownership,

- integration patterns,
- and technology constraints.

In an AI-driven commerce setting, the architectural question is not only “which model should we use?” It is also:

- Which systems generate the data that the model depends on?
- Which domain owns the business decision that the model influences?
- Which workflow must remain reliable when the model is unavailable or uncertain?
- How are predictions turned into operational actions in storefront, CRM, ERP, or support systems?

5.3 Enterprise architecture as a set of views

An architecture description is easier to manage when it is divided into views. For e-commerce, four views are especially practical:

- **Business architecture:** capabilities, value streams, organization roles, policies, and business outcomes.
- **Application architecture:** major systems, services, platform modules, ownership boundaries, and interactions.
- **Data architecture:** entities, identifiers, systems of record, analytical pipelines, and governance rules.
- **Technology architecture:** hosting environment, integration infrastructure, observability stack, identity/security, and platform services.

These views should not be treated as isolated documents. They are projections of one system. For example, a business capability such as pricing must connect to application services, authoritative data sources, and runtime controls.

5.4 Capability maps

A capability map describes what an organization must be able to do, independent of how the capability is implemented today. That distinction matters because teams, products, and vendor choices can change while core business abilities remain relatively stable.

5.4.1 Core e-commerce capabilities

An AI-enabled commerce organization often needs capabilities such as:

- customer acquisition and campaign orchestration,
- product information and catalog management,
- search, discovery, and merchandising,
- personalization and recommendation,
- pricing, promotions, and offer management,
- cart, checkout, and payment orchestration,
- order management and fulfillment coordination,
- returns, refunds, and post-purchase support,
- customer profile and loyalty management,
- analytics, experimentation, and decision support,
- security, privacy, fraud, and governance.

5.4.2 Why capability maps are useful

Capability maps help teams avoid jumping directly into solution design. For example, if an organization says “we need a recommendation engine,” the architectural question should be expanded:

- Is personalization a standalone capability, or is it part of a broader discovery capability?
- Which parts of the recommendation workflow belong to the storefront experience layer, and which belong to a shared data platform?
- Which existing operational capabilities must change to make recommendations effective, such as promotions, inventory awareness, or content quality?

Capability maps also expose imbalance. A company may invest heavily in customer acquisition and storefront design while underinvesting in returns, data governance, or seller operations. Architecture should reveal such weaknesses before they become scaling problems.

5.5 Value streams

Capabilities describe *what* the organization can do. Value streams describe *how value moves* through the business from trigger to outcome.

5.5.1 Typical e-commerce value streams

Important value streams include:

- **Browse-to-buy:** discovery, consideration, cart, checkout, payment, confirmation.
- **Order-to-cash:** order capture, validation, fulfillment, invoicing, settlement, financial posting.
- **Procure-to-stock:** replenishment planning, supplier ordering, receiving, storage, inventory updates.
- **Return-to-resolution:** return request, approval, pickup/drop-off, inspection, refund or exchange.
- **Lead-to-account:** acquisition, registration, qualification, first purchase, retention.

5.5.2 Why value streams matter architecturally

Value streams expose cross-domain handoffs. For example, “browse-to-buy” may touch search, catalog, pricing, promotions, checkout, payment, fraud, order management, and analytics. If the stream depends on unclear ownership or duplicated business rules, failures appear at handoff points rather than inside a single system.

Value-stream mapping is especially valuable in hybrid commerce environments where storefront systems and ERP systems divide responsibility. Students should learn to ask not only which system executes a step, but which system is accountable for the business meaning of that step.

5.6 Domain boundaries and bounded contexts

Large commerce platforms become difficult to scale when everything is treated as one application or one shared database. Architecture therefore needs domain boundaries with explicit ownership.

5.6.1 Typical commerce domains

Common domains include:

- **Catalog:** products, attributes, categories, media, content enrichment.
- **Pricing and promotions:** price lists, discount rules, coupon logic, campaign offers.
- **Inventory and availability:** stock position, reservations, safety stock, availability promises.
- **Cart and checkout:** session basket state, shipping options, tax estimation, checkout workflow.
- **Payments:** payment intents, authorization, capture, refund orchestration.
- **Orders:** order lifecycle, status management, amendments, cancellations.

- **Customer and identity:** profile, authentication, preferences, loyalty, segmentation.
- **Fulfillment and logistics:** warehouse tasks, shipment creation, delivery events, returns routing.
- **Analytics and experimentation:** event collection, metric definitions, experiment assignment.

5.6.2 Good boundaries versus bad boundaries

A good boundary reduces coupling and clarifies ownership of behavior and data. A poor boundary creates ambiguity such as:

- pricing rules duplicated in storefront, ERP, and marketing tools,
- customer attributes updated independently in CRM and commerce applications,
- order status interpreted differently by warehouse, finance, and support teams,
- product IDs that differ between catalog, analytics, and inventory systems.

Architectural boundaries should therefore be tested against real business scenarios. If a business change requires editing logic in five systems with no clear owner, the boundary is likely wrong.

5.7 Systems of record and systems of engagement

Not every system should be authoritative for every type of data. One of the most important EA concepts in commerce is deciding the *system of record* for each key entity.

5.7.1 Typical patterns

- The **commerce layer** often owns high-velocity interaction data such as carts, storefront sessions, and recommendation exposures.
- The **ERP layer** often owns invoices, financial postings, stock movements, supplier records, and contractual B2B customer data.
- A **PIM or catalog service** may own enriched product content while ERP owns procurement-oriented product master fields.
- A **data platform** may own analytical aggregates and features but should not silently become the operational source of truth.

5.7.2 Systems of engagement

Storefronts, mobile apps, support consoles, and seller portals are often systems of engagement rather than systems of record. They present, collect, and orchestrate information, but they may not be authoritative for final commercial truth. Confusing these roles leads to reconciliation failures.

For example, a storefront may display an order as “confirmed” while ERP has not yet accepted it due to credit rules or stock constraints. Architecture must define which state is customer-facing, which state is financially binding, and how transitions are synchronized.

5.8 Reference architecture for AI-driven e-commerce

There is no single universal architecture, but a practical reference pattern often includes the following layers:

- **Experience layer:** web, mobile, agent interfaces, seller/admin portals.
- **Commerce services layer:** catalog, search, pricing, cart, checkout, order orchestration, customer services.
- **Enterprise systems layer:** ERP, CRM, WMS, finance, procurement, supplier management.
- **Integration layer:** APIs, event streams, workflow orchestration, master-data synchronization.
- **Data and AI layer:** event collection, warehouse/lakehouse, feature pipelines, experimentation, model training, model serving.
- **Cross-cutting controls:** identity, security, privacy, audit, observability, governance.

5.8.1 Where AI fits

AI should be mapped to business capabilities rather than inserted as an isolated layer that “does intelligence.” Examples include:

- search ranking and recommendations within discovery,
- fraud scoring within checkout and payments,
- demand forecasting within inventory and replenishment,
- case summarization and response assistance within support,
- anomaly detection within operations and finance.

This mapping matters because it keeps accountability clear. The discovery domain owns business outcomes for search and recommendation even if the underlying models are developed by a centralized data team.

5.9 Current state, target state, and transition architecture

Architecture is not useful if it only documents the present. Organizations need a target state that guides investment and sequencing.

5.9.1 Current-state analysis

Current-state architecture should identify:

- duplicated systems and overlapping responsibilities,
- brittle integrations and manual reconciliation steps,
- data ownership conflicts,
- missing capabilities,
- and operational pain points such as latency, outages, or inconsistent KPIs.

5.9.2 Target architecture

Target architecture is a future-state design that expresses intended business and technical structure. It is not a vendor brochure and not a line-by-line implementation plan. It should answer:

- Which domains and systems will own which capabilities?
- Which integrations will be synchronous, asynchronous, or batch?
- Which entities will be mastered where?
- Which platforms are shared across business units?
- Which AI capabilities will be productized versus embedded locally?

5.9.3 Transition architecture

Most organizations cannot move directly from current state to target state. Transition architecture defines intermediate steps. Examples include:

- separating catalog and pricing ownership before replacing checkout,
- introducing a shared event stream before centralizing experimentation,
- establishing a customer identity service before rolling out cross-channel personalization.

This is important in enterprise settings because a theoretically elegant target architecture may fail if the migration path is not operationally realistic.

5.10 Architectural concerns specific to AI

AI adds architectural concerns beyond model selection. Representative concerns include:

- **Training-serving consistency:** features must mean the same thing offline and online.
- **Latency budgets:** recommendation or fraud decisions may need sub-second responses.
- **Fallback behavior:** the commerce workflow must continue safely if a model is unavailable.
- **Observability:** systems must track not only uptime but also prediction quality, drift, and business impact.
- **Governance:** model changes may alter business policy and therefore require approval, auditability, and rollback procedures.

In many organizations, the failure is not that models are weak. The failure is that model outputs are not architecturally integrated into business workflows with clear ownership and control points.

5.11 Operating model and team topology

Architecture is influenced by team structure. A domain with clear technical boundaries but no accountable product owner often performs badly in practice.

5.11.1 Typical team patterns

- **Domain-aligned product teams:** own specific business capabilities such as checkout or catalog.
- **Platform teams:** provide shared infrastructure such as identity, observability, experimentation, or data platforms.
- **Enterprise systems teams:** maintain ERP, CRM, accounting, and operational master-data platforms.
- **Central AI or data teams:** develop shared methods, governance, and model platforms while collaborating with domain teams on use cases.

An architecture is more likely to succeed when ownership in the diagrams matches ownership in the organization. If a system is “owned by everyone,” it is usually owned by no one.

5.12 Artifacts for students and architects

A minimal EA artifact set for an e-commerce program may include:

- a capability map,
- one or more value stream maps,
- a domain map with ownership boundaries,
- a system-context diagram,
- a system-of-record matrix for key entities,
- and a target-state architecture note with transition priorities.

These artifacts should be consistent with each other. If the domain map says pricing is a shared domain but the system-of-record matrix shows three authoritative pricing systems, the architecture still contains a contradiction.

5.13 Hands-on artifacts

The key deliverable for this chapter is a compact but defensible architecture package:

- an e-commerce capability map,
- a domain model showing clear ownership,
- and a short target-architecture note that explains systems of record and integration choices.

5.14 Managerial implications

Enterprise architecture is sometimes dismissed as documentation overhead. In e-commerce, that is a mistake. Architectural ambiguity produces real costs:

- slow product delivery because every change triggers cross-team negotiation,
- financial reconciliation issues from conflicting commercial truth,
- duplicated AI initiatives that solve the same problem inconsistently,
- and weak governance when no domain clearly owns a high-impact decision.

Good architecture does not eliminate complexity. It makes complexity explicit, owned, and therefore manageable.

5.15 Multiple-choice questions (MCQs)

1. In enterprise architecture, a *capability map* primarily describes:
 - (a) A list of specific microservices and their endpoints.
 - (b) *What* the business does (stable abilities), independent of *how* it is implemented.
 - (c) The physical network topology of the data center.
 - (d) A Gantt chart of the project plan.
2. A good domain boundary (e.g., for a “Catalog” domain) typically aims to:
 - (a) Maximize shared database tables across all teams.
 - (b) Minimize coupling and define clear ownership of data and behaviors.
 - (c) Ensure all operations are synchronous.
 - (d) Avoid having APIs.
3. Which artifact is most suitable for describing how business value is delivered end-to-end?
 - (a) Value stream map
 - (b) Entity-relationship diagram (ERD)
 - (c) Source code repository layout
 - (d) TLS configuration file
4. “Target architecture” is best described as:
 - (a) The current-state system diagram.
 - (b) A future-state design that guides decisions and sequencing of change.
 - (c) A vendor product brochure.
 - (d) A test plan for unit tests.
5. In an AI-driven e-commerce context, which is the best example of an *architectural concern* (not a model choice)?
 - (a) Whether to use XGBoost or logistic regression
 - (b) How to ensure training-serving consistency and monitor drift
 - (c) Whether to use k -means or DBSCAN
 - (d) Whether to use SGD or Adam

Answer key

1. (b)
2. (b)
3. (a)
4. (b)
5. (b)

5.16 Exercises (short)

1. Draft a capability map for an AI-enabled e-commerce organization. Include at least: customer acquisition, discovery, pricing/promotions, order management, fulfillment/returns, customer support, data/analytics, and governance/security.
2. Choose one value stream (e.g., “order-to-cash” or “browse-to-buy”). Identify 5–8 steps and list the owning domain/system for each step (commerce vs ERP vs logistics vs payment).
3. Propose domain boundaries for: catalog, pricing, orders, payments, and customer profiles. For each boundary, name the system-of-record for key data entities.

5.17 Mini-case (Odoo-linked, vendor-neutral)

Scenario: Your company is hybrid B2C/B2B. Odoo is used for core ERP flows (Sales, Inventory, Accounting, CRM). A separate commerce layer handles web/mobile experiences and AI features.

Task: Produce a short “target architecture” note:

- Draw a capability-to-system mapping (capabilities → commerce layer vs Odoo vs data platform).
- Define the *system of record* for customer, product, price list, order, invoice, and shipment.
- Identify 3 integration risks (data consistency, latency, duplicate sources of truth) and propose mitigations.

Chapter 6

Enterprise Integration Patterns for E-Commerce (Week 5)

6.1 Scope (placeholder)

- API-first, integration styles (sync/async), event-driven architecture.
- iPaaS/ESB concepts, message brokers/streams, and reliability patterns.

6.2 Hands-on lab (placeholder)

- Model an order lifecycle with events + idempotency + outbox pattern (conceptual + pseudo).

6.3 Multiple-choice questions (MCQs)

1. Which statement best characterizes synchronous vs asynchronous integration?
 - (a) Synchronous integration cannot fail; asynchronous always fails.
 - (b) Synchronous integration couples availability and latency; asynchronous trades immediacy for resilience and decoupling.
 - (c) Asynchronous integration is only for analytics; synchronous is only for ERP.
 - (d) They are equivalent as long as JSON is used.
2. Why is idempotency important in order processing APIs?
 - (a) It makes responses larger.
 - (b) It prevents duplicate effects when requests are retried.
 - (c) It eliminates the need for authentication.
 - (d) It guarantees exactly-once message delivery in all systems.
3. The *outbox pattern* is primarily used to:

- (a) Store images for the catalog.
 - (b) Reliably publish events when database writes succeed, avoiding dual-write inconsistencies.
 - (c) Encrypt API traffic.
 - (d) Replace a message broker.
4. In event-driven systems, a *consumer* should typically be designed to be:
- (a) State-less but non-retryable
 - (b) Idempotent and retryable
 - (c) Dependent on message ordering always being perfect
 - (d) Tightly coupled to producer database schemas
5. Which is a common reason to choose events over direct API calls for some flows?
- (a) To avoid defining schemas/contracts
 - (b) To decouple systems and allow multiple downstream consumers without changing the producer
 - (c) To guarantee zero latency
 - (d) To avoid monitoring

Answer key

- 1. (b)
- 2. (b)
- 3. (b)
- 4. (b)
- 5. (b)

6.4 Exercises (short)

- 1. For each flow, choose sync API or async events and justify: (i) “place order”, (ii) “update inventory availability”, (iii) “invoice posted”, (iv) “shipment delivered”, (v) “customer profile updated”.
- 2. Design an idempotency strategy for the “create order” API. Specify the idempotency key and how long it is retained.
- 3. Write a short failure story: a dual-write bug between database and message broker. Explain how the outbox pattern prevents it.

6.5 Mini-case (Odoo-linked, vendor-neutral)

Scenario: A headless storefront integrates with Odoo for orders, inventory, invoices, and CRM updates.

Task: Propose an integration design that supports scale and reliability:

- Define the key domain events (at least 8) and which system publishes each.
- Identify where you need synchronous APIs (e.g., payment authorization) vs asynchronous events (e.g., shipment updates).
- Specify two data contracts and a versioning strategy to avoid breaking changes.
- List the minimum observability signals (logs/metrics/traces) required to operate the integration in production.

Chapter 7

ERP/CRM/SCM Integration in Commerce (Week 6)

7.1 Scope (placeholder)

- Odoo overview (high-level): key modules relevant to commerce (Sales, Inventory, Accounting, CRM).
- Order-to-cash, inventory, fulfillment, returns, and customer data flows.
- Master data management (MDM) and data ownership across domains.

7.2 Artifacts (placeholder)

- Deliverable: integration context diagram + data contracts (high level), including an Odoo boundary.

7.3 Multiple-choice questions (MCQs)

1. In an integrated commerce + ERP architecture, the *system of record* is best defined as:
 - (a) The system with the nicest UI
 - (b) The system that is most convenient for analytics
 - (c) The authoritative source of truth for a given entity (e.g., invoice) and its lifecycle
 - (d) The system that runs in the cloud
2. Which flow is most strongly associated with *order-to-cash*?
 - (a) Creating a product image embedding
 - (b) Quote/order → delivery → invoice → payment reconciliation
 - (c) Building a recommendation model

- (d) Defining a Kafka topic
- 3. Master data management (MDM) is primarily concerned with:
 - (a) Training deep learning models for search
 - (b) Defining and governing shared entities (customer, product, supplier) and preventing duplicates/inconsistencies
 - (c) Choosing the best database index
 - (d) Encrypting API requests
- 4. A common integration risk when connecting commerce with ERP is:
 - (a) Too many unit tests
 - (b) Duplicate sources of truth for prices, customers, or inventory
 - (c) Having an event schema
 - (d) Using idempotency keys
- 5. In practice, why do integrations often require both APIs and events?
 - (a) Because events are always cheaper than APIs
 - (b) Because some steps require immediate confirmation while others benefit from decoupling and retries
 - (c) Because ERP systems cannot expose APIs
 - (d) Because events guarantee zero duplicates without design effort

Answer key

- 1. (c)
- 2. (b)
- 3. (b)
- 4. (b)
- 5. (b)

7.4 Exercises (short)

- 1. For each entity, choose a system of record (commerce vs Odoo) and justify: customer, product, price list, inventory on-hand, order, invoice, payment, return.
- 2. Describe two failure modes of inventory sync and propose mitigations (e.g., eventual consistency, reservations, reconciliation jobs).
- 3. Draft a “data contract” for an `OrderCreated` event: required fields, optional fields, and versioning rules.

7.5 Mini-case (Odoo-linked, vendor-neutral)

Scenario: A hybrid company runs a headless storefront for B2C and a B2B portal for wholesale customers. Odoo is used for Sales, Inventory, Accounting, and CRM.

Task: Design the integration for three end-to-end processes:

- **Order-to-cash:** from checkout to invoice posting and payment reconciliation.
- **Returns/refunds:** from return request to stock adjustment and credit note.
- **B2B pricing:** negotiated price lists and approvals without duplicating sources of truth.

For each process, state: key events, required synchronous calls, idempotency strategy, and the minimum monitoring you would implement.

Chapter 8

Recommenders I: Classical Methods + Learning-to-Rank (Week 7)

8.1 Scope (placeholder)

- Collaborative filtering, matrix factorization, and implicit feedback.
- Learning-to-rank: pointwise/pairwise/listwise framing; offline vs online evaluation.

8.2 Hands-on lab (placeholder)

- Baseline recommender with clear metrics (MAP@K/NDCG@K).

Chapter 9

Recommenders II: Deep Learning, Embeddings, and Retrieval (Week 8)

9.1 Scope (placeholder)

- Two-tower retrieval, embeddings, sequence models, and candidate generation vs re-ranking.
- Practical concerns: cold-start, diversity, bias, and latency.

9.2 Hands-on lab (placeholder)

- Train embeddings; build a retrieval + re-ranking pipeline (high level).

Chapter 10

Pricing and Promotions with ML + Optimization (Week 9)

10.1 Scope (placeholder)

- Demand forecasting, elasticity, uplift, and constraints (inventory/competition).
- Optimization approaches and experiment design for pricing.

10.2 Hands-on lab (placeholder)

- Forecast baseline + simple constrained optimization scenario.

Chapter 11

Fraud, Risk, and Trust & Safety (Week 10)

11.1 Scope (placeholder)

- Fraud types: payment, account takeover, promotion abuse, refund abuse.
- Methods: supervised learning, anomaly detection, graph ML, and rule+ML hybrids.

11.2 Hands-on lab (placeholder)

- Build a risk scoring baseline + thresholding and cost-sensitive evaluation.

Chapter 12

Customer Service Automation with LLMs (Week 11)

12.1 Scope (placeholder)

- RAG, tool use, workflow orchestration, evaluation, and guardrails.
- When to use LLMs vs classical NLP vs rules.

12.2 Hands-on lab (placeholder)

- Prototype: FAQ assistant with retrieval + citations (local documents only).

Chapter 13

MLOps + DataOps for E-Commerce (Week 12)

13.1 Scope (placeholder)

- Platform-agnostic MLOps/DataOps: CI/CD for ML, pipelines, and reproducibility.
- Feature stores (concept), training-serving skew, monitoring, drift, and SLOs.
- Experiment tracking, model governance, and evaluation at scale.

13.2 Artifacts (placeholder)

- Deliverable: ML system design doc + monitoring plan.

Chapter 14

Security, Privacy, Compliance, and Responsible AI (Week 13)

14.1 Scope (placeholder)

- Payments/security basics, privacy-by-design, and data minimization.
- Threat modeling for e-commerce and AI features (LLM-specific risks included).
- Responsible AI: fairness, explainability, auditability, and human-in-the-loop controls.

14.2 Discussion (placeholder)

- Risk assessment template for an AI feature.

Chapter 15

Capstone: Enterprise-Grade AI E-Commerce Architecture (Week 14)

Capstone scenario (agreed)

- Scenario: end-to-end “AI commerce + Odoo” reference architecture for a single company.
- Goal: demonstrate EA alignment, ERP integration, and measurable AI value with operational readiness.
- Business model: hybrid (B2C + B2B).
- Commerce front-end: both (compare Odoo Website/eCommerce vs a separate headless storefront integrating to Odoo).
- AI stack: both (ML for recs/fraud/pricing + LLMs for support/ops).

15.1 Scope (placeholder)

- End-to-end architecture: channels, services, data platform, ML/LLM layer, ERP integration.
- Evaluation: business metrics, technical metrics, and operational readiness.

15.2 Capstone deliverables (placeholder)

- Architecture pack (diagrams), backlog, data contracts, and evaluation plan.

Appendix A

Datasets and Synthetic Data for Teaching (placeholder)

Appendix B

Template: System Design Document (placeholder)

Appendix C

Template: Data Contract (placeholder)

Appendix D

Template: Model Card and Risk Assessment (placeholder)